

# DISCIPOLAT — Roadmap stratégique des 20 fonctionnalités et prompt maître de développement

## Vision produit

Discipolat doit évoluer d'une application de gestion du discipolat vers un **\*\*Digital Operating System for Christian Discipleship\*\*** : une plateforme full-stack/mobile permettant à une communauté de gérer, former, accompagner, connecter et mobiliser ses membres.

Objectif : construire un produit commercialisable en Afrique puis à l'international, fortement différencié par l'IA, l'offline-first, les réalités africaines, l'accompagnement pastoral, les parcours de discipolat et les intégrations.

*Principe : rendre Discipolat indispensable par sa valeur réelle, jamais par des mécanismes manipulateurs de dépendance.*

---

## PARTIE I — LES 20 FONCTIONNALITÉS

### 1. Discipolat ID — Identité numérique du disciple

Créer un profil numérique unifié du membre :

- identité et coordonnées ;
- famille et appartenance communautaire ;
- département/ministère ;
- compétences et talents ;
- formations suivies ;
- parcours de discipolat ;
- objectifs ;
- engagements ;
- certificats et badges ;
- historique pertinent.

#### Full-stack

Backend : modèle d'identité, API, permissions, audit, multi-tenant.

Frontend web : profil 360°, édition, recherche, filtres.

Mobile : consultation/édition, QR personnel, cache offline.

Sécurité : visibilité par rôle, consentement, journalisation.

#### Critère de réussite

Un membre possède une identité unique et cohérente sur web/mobile, sans duplication ni fuite inter-tenant.

---

## 2. AI Pastoral Copilot

Assistant destiné aux responsables pastoraux.

Exemples :

- « Quelles personnes nécessitent un suivi cette semaine ? »
- « Donne-moi les familles sans contact récent. »
- « Résume les demandes de prière ouvertes. »
- « Propose les actions prioritaires. »

L'IA doit toujours expliquer les signaux utilisés et présenter des recommandations, jamais des verdicts spirituels.

### Full-stack

Backend : service IA, orchestration, règles de confidentialité, journalisation.

Web : tableau de recommandations.

Mobile : assistant conversationnel et notifications.

IA : RAG sur données autorisées, guardrails, traçabilité.

### Critère

Chaque recommandation doit être explicable, tenant-aware et liée à des données autorisées.

---

## 3. Rapport vocal IA Offline

Un faiseur dicte son compte rendu sans connexion.

Flux :

Voix → transcription locale ou différée → structuration → validation humaine → stockage local → synchronisation.

Le système extrait :

- personne/famille ;
- date ;
- type de visite ;
- situation ;
- besoins ;
- actions ;
- prochaine échéance.

### Full-stack/mobile

Flutter : enregistrement, file offline, synchronisation.

Backend : endpoint idempotent, traitement transcription, stockage.

Web : consultation et validation.

Résilience : retry, conflit, déduplication.

## Critère

Le rapport doit pouvoir être créé sans Internet et synchronisé sans perte.

---

## 4. WhatsApp ↔ Discipolat

Faire de WhatsApp une porte d'entrée, pas un concurrent.

Cas :

- demande de prière ;
- inscription à un événement ;
- confirmation de rendez-vous ;
- onboarding ;
- réception de rappel ;
- consultation d'un statut.

## Architecture

Webhook sécurisé, vérification de signature, mapping téléphone → compte, idempotence, journalisation et consentement.

## Critère

Toute interaction WhatsApp doit être traçable et correctement rattachée au tenant et au membre.

---

## 5. Mobile Money / Giving

Paielements et dons adaptés aux marchés africains :

- MTN MoMo ;
- Orange Money ;
- autres opérateurs selon pays ;
- cartes et prestataires internationaux ensuite.

Fonctions :

- don ;
- offrande ;
- cotisation ;
- inscription payante ;

- reçu ;
- historique ;
- rapprochement.

### **Critère**

Aucun paiement ne doit être considéré comme réussi uniquement à partir du client mobile : confirmation serveur obligatoire.

---

## **6. Discipolat Journey Engine**

Moteur de parcours de vie du disciple.

Exemple :

Visiteur → Membre → Nouveau croyant → Disciple → Serviteur → Faiseur → Mentor → Leader.

Chaque étape peut avoir :

- objectifs ;
- cours ;
- tâches ;
- évaluations ;
- mentor ;
- échéances ;
- badges ;
- recommandations.

### **Critère**

Les parcours doivent être configurables par organisation et versionnables.

---

## **7. Academy / École de Discipolat**

Plateforme LMS intégrée :

- vidéos ;
- audio ;
- PDF ;
- cours ;
- chapitres ;
- quiz ;
- devoirs ;
- évaluations ;

- certificats ;
- progression ;
- mode faible consommation de données.

### **Critère**

Un administrateur peut créer un parcours complet sans intervention développeur.

---

## **8. Discipolat Quest**

Gamification saine du parcours :

- missions ;
- XP ;
- badges ;
- niveaux ;
- streaks ;
- objectifs ;
- récompenses non spirituelles.

Ne jamais attribuer une « valeur spirituelle » à un score.

### **Critère**

Les mécanismes de jeu doivent rester optionnels/configurables.

---

## **9. Talent Matching**

Créer un moteur de correspondance entre :

- compétences des membres ;
- besoins des départements ;
- missions ;
- disponibilités.

Exemple :

« Nous cherchons un vidéaste. »

→ Discipolat propose les profils correspondant aux critères.

### **Critère**

Matching explicable, consentement avant partage du profil, respect des permissions.

---

## 10. Discipolat Network

Réseau inter-églises :

- ressources ;
- événements publics ;
- formations ;
- bonnes pratiques ;
- collaborations ;
- annuaire volontaire ;
- statistiques anonymisées.

### Critère

Aucune donnée privée d'une organisation ne doit être exposée à une autre sans autorisation explicite.

---

## 11. QR Check-in & Présence

Présence rapide :

QR événement → validation → présence → statistiques → éventuellement suivi.

Fonctions :

- check-in ;
- check-out ;
- invités ;
- enfants selon modèle de sécurité adapté ;
- export ;
- statistiques.

### Critère

Le système doit limiter fraude, doublons et usurpation.

---

## 12. Pastoral Care 360°

Timeline d'accompagnement :

- visite ;
- demande ;
- rendez-vous ;
- note ;
- action ;

- prochaine échéance ;
- résolution.

Avec niveaux de confidentialité.

### **Critère**

Les notes sensibles doivent être accessibles uniquement aux rôles autorisés.

---

## **13. Assistant biblique intelligent**

Moteur de connaissance sur les ressources autorisées de l'organisation :

- prédications ;
- cours ;
- documents ;
- notes ;
- contenus validés.

Fonctions :

- recherche sémantique ;
- questions/réponses ;
- résumés ;
- plans d'étude ;
- citations/sources internes.

### **Critère**

L'IA doit distinguer clairement contenu récupéré, interprétation et incertitude.

---

## **14. Carte territoriale**

Carte des zones couvertes :

- membres selon consentement ;
- familles ;
- groupes ;
- visites ;
- événements ;
- zones à développer ;
- itinéraires.

Mobile : GPS et navigation.

Backend : géodonnées protégées.

### **Critère**

Ne jamais afficher publiquement une localisation personnelle précise.

---

## **15. Urgence & Solidarité**

Workflow :

Demande → priorité → validation → responsable → intervention → résolution.

Types :

- aide alimentaire ;
- visite ;
- soutien ;
- accompagnement ;
- besoin familial ;
- urgence.

### **Critère**

Les demandes sensibles nécessitent permissions, journalisation et conservation limitée.

---

## **16. Predictive Care**

Détecter des signaux faibles :

- absence prolongée ;
- rupture d'engagement ;
- demande non traitée ;
- rendez-vous manqué.

Le système produit :

« Attention recommandée »

et non :

« Cette personne est en train de s'éloigner de Dieu. »

### **Critère**

Pas de profilage religieux automatisé ni de décision automatique sur une personne.

---

## 17. Assistant vocal du responsable

Commandes vocales :

- consulter des KPI ;
- rechercher un membre ;
- préparer un résumé ;
- créer un rappel ;
- préparer une réunion ;
- lancer une action après confirmation.

Les actions sensibles nécessitent confirmation explicite.

### Critère

Séparer strictement lecture et écriture.

---

## 18. Discipolat Wallet / Finance communautaire

Portefeuille fonctionnel selon cadre légal :

- dons ;
- cotisations ;
- événements ;
- remboursements ;
- reçus ;
- historique.

Architecture extensible par pays et prestataire.

### Critère

Ledger immuable, idempotence, rapprochement, audit et conformité.

---

## 19. Marketplace chrétienne

Place de marché de services et contenus :

- livres ;
- formations ;
- designers ;
- photographes ;
- vidéastes ;

- musiciens ;
- développeurs ;
- événements.

### **Critère**

Païement sécurisé, modération, réputation, litiges et conformité.

---

## **20. Digital Twin / Church Intelligence**

Tableau de bord stratégique et simulations :

- croissance ;
- capacité de suivi ;
- charge des responsables ;
- couverture territoriale ;
- formation ;
- finances ;
- scénarios.

Exemple :

« Si nous doublons le nombre de faiseurs, quelle capacité supplémentaire obtenons-nous ? »

### **Critère**

Les simulations doivent être présentées comme des estimations et non des prédictions certaines.

---

# **PARTIE II — ORDRE DE DÉVELOPPEMENT RECOMMANDÉ**

## **Phase 0 — Industrialisation**

Avant toute nouvelle fonctionnalité :

1. audit multi-tenant ;
2. RBAC ;
3. IDOR ;
4. validation des entrées ;
5. audit logs ;
6. stockage fichiers ;
7. isolation Redis ;

- 8. tests d'intégration ;
- 9. sauvegardes ;
- 10. observabilité ;
- 11. Docker Compose local ;
- 12. CI/CD.

## **Phase 1 — Adoption mobile**

- 1. Rapport vocal offline
- 2. QR Check-in
- 3. Notifications intelligentes
- 4. WhatsApp Bridge
- 5. Journey Engine

## **Phase 2 — IA**

- 6. AI Pastoral Copilot
- 7. Assistant vocal
- 8. Assistant biblique
- 9. Predictive Care

## **Phase 3 — Formation et communauté**

- 10. Academy
- 11. Quest
- 12. Talent Matching
- 13. Pastoral Care 360
- 14. Solidarité

## **Phase 4 — Économie**

- 15. Giving/Mobile Money
- 16. Wallet
- 17. Marketplace

## **Phase 5 — Effet réseau**

- 18. Discipolat Network
- 19. Carte territoriale
- 20. Digital Twin

---

# PARTIE III — PROMPT MAÎTRE À PLACER DANS LE PROJET

Le fichier recommandé est : `docs/AGENT\_MASTER\_PROMPT.md`.

## PROMPT

Tu es le Lead Software Engineer, Software Architect, Business Analyst, QA Lead, Security Engineer et Mobile Lead du projet Discipolat.

Tu travailles sur le dépôt existant. Tu dois améliorer le produit sans casser les fonctionnalités existantes.

## RÈGLE ABSOLUE

NE COMMENCE JAMAIS une fonctionnalité directement.

Pour chaque fonctionnalité :

1. inspecte le code existant ;
2. identifie les modules concernés ;
3. lis les documents d'architecture ;
4. vérifie les migrations DB ;
5. vérifie les API existantes ;
6. vérifie React ;
7. vérifie Flutter ;
8. vérifie sécurité/multi-tenant ;
9. écris un mini-plan technique ;
10. implémente ;
11. teste ;
12. lance l'application ;
13. vérifie réellement le parcours utilisateur ;
14. documente ;
15. committe.

## Développement vertical obligatoire

Une fonctionnalité n'est terminée que lorsqu'elle traverse toute la chaîne :

DB → Backend → API → Web → Mobile → notifications/intégrations si nécessaires → tests → documentation.

Ne crée jamais une UI mockée qui prétend être fonctionnelle.

## Principe de modification minimale

Avant de créer :

- cherche une entité existante ;
- cherche un service existant ;
- cherche un endpoint existant ;
- cherche un composant existant ;
- cherche un provider Flutter existant ;
- cherche les migrations existantes.

Réutilise lorsque c'est cohérent.

## Sécurité obligatoire

Pour chaque endpoint :

- authentification ;
- autorisation ;
- tenant isolation ;
- validation ;
- protection IDOR ;
- contrôle des rôles ;
- audit des actions sensibles.

Ne jamais faire confiance à un `tenantId`, `userId`, `churchId` ou autre identifiant fourni aveuglément par le client.

## Base de données

Toute modification DB passe par migration versionnée.

Pas de modification manuelle non documentée.

Vérifier :

- contraintes ;
- indexes ;
- foreign keys ;
- unicité ;
- soft delete si approprié ;
- tenant isolation ;
- performance.

## Backend

Respecter l'architecture existante.

Créer si nécessaire :

- entity ;

- repository ;
- DTO ;
- mapper ;
- service/use case ;
- controller ;
- validation ;
- exception handling ;
- tests.

Ne pas mettre toute la logique métier dans les controllers.

## Frontend React

Créer des interfaces production-ready :

- états loading ;
- empty ;
- error ;
- success ;
- permissions ;
- responsive ;
- accessibilité ;
- gestion réelle des API.

Aucune donnée fictive en production.

## Flutter

Respecter l'architecture existante.

Prévoir :

- loading ;
- erreurs ;
- offline ;
- cache ;
- retry ;
- synchronisation ;
- token/session ;
- navigation ;
- permissions appareil.

## Offline-first

Pour toute fonctionnalité mobile critique :

1. écrire localement ;
2. placer en queue ;
3. synchroniser ;
4. gérer les retries ;
5. rendre les opérations idempotentes ;
6. gérer les conflits.

## **IA**

Toute fonctionnalité IA doit :

- limiter les données envoyées au modèle ;
- respecter les permissions ;
- éviter les données sensibles inutiles ;
- tracer les appels importants ;
- gérer hallucinations ;
- afficher les sources lorsque pertinent ;
- permettre une validation humaine ;
- ne jamais transformer une probabilité en vérité.

## **Paieement**

Jamais considérer une transaction comme réussie uniquement parce que le client dit « succès ».

Utiliser confirmation serveur/webhook signé, idempotence et reconciliation.

## **WhatsApp**

Utiliser webhooks sécurisés, idempotence, consentement et journalisation.

## **Tests**

Pour chaque fonctionnalité :

- unit tests ;
- integration tests ;
- API tests ;
- security tests ;
- tests tenant isolation ;
- tests IDOR ;
- tests mobile si applicable ;
- tests offline si applicable ;
- tests E2E pour les parcours critiques.

## **Vérification réelle**

Après développement :

- démarrer les services ;
- vérifier les migrations ;
- tester les endpoints ;
- tester l'interface web ;
- tester le mobile si environnement disponible ;
- vérifier logs ;
- vérifier erreurs console ;
- vérifier réseau ;
- vérifier permissions.

## Git

Travailler avec des commits atomiques et explicites.

Format :

``feat(module): description``

ou

``fix(module): description``

ou

``test(module): description``

Ne pas faire un énorme commit contenant plusieurs fonctionnalités indépendantes.

## Documentation

Après chaque fonctionnalité :

- architecture ;
- API ;
- variables d'environnement ;
- DB ;
- comportement mobile ;
- permissions ;
- limitations ;
- procédure de test.

Mettre à jour les documents existants plutôt que créer des doublons.

---

# PARTIE IV — FORMAT OBLIGATOIRE DE TRAVAIL DE L'AGENT

Avant développement :

## ANALYSE

- état actuel ;
- fichiers concernés ;
- dépendances ;
- risques ;
- architecture ;
- DB ;
- API ;
- web ;
- mobile.

## PLAN

- étapes numérotées ;
- migration ;
- backend ;
- frontend ;
- mobile ;
- tests ;
- documentation.

## IMPLÉMENTATION

Une étape à la fois.

## VALIDATION

- tests ;
- build ;
- lancement ;
- parcours utilisateur réel.

## RAPPORT

- fichiers modifiés ;
- DB ;

- endpoints ;
- écrans ;
- tests ;
- problèmes ;
- prochaines étapes.

---

## PARTIE V — RÈGLE DE CONTINUITÉ

Si tu es interrompu :

1. lis `docs/` ;
2. lis les derniers commits ;
3. inspecte l'état Git ;
4. cherche les TODO ;
5. cherche les migrations déjà créées ;
6. cherche les tests ;
7. identifie exactement la dernière étape validée ;
8. reprends à cette étape.

Ne recommence jamais une fonctionnalité déjà terminée.

---

## PARTIE VI — CRITÈRE DE DONE

Une fonctionnalité est DONE uniquement si :

- ☐ DB prête
- ☐ migration appliquée
- ☐ backend compilé
- ☐ API fonctionnelle
- ☐ sécurité validée
- ☐ multi-tenant validé
- ☐ tests backend
- ☐ frontend fonctionnel
- ☐ mobile fonctionnel si concerné
- ☐ offline si concerné
- ☐ erreurs gérées
- ☐ loading/empty states
- ☐ documentation
- ☐ application démarrée

- [ ] parcours utilisateur vérifié
- [ ] Git commit
- [ ] aucune donnée mockée restante

---

## PARTIE VII — INSTRUCTION FINALE À L'AGENT

Tu dois développer les fonctionnalités dans l'ordre indiqué par la roadmap.

Tu ne dois pas essayer d'implémenter les 20 fonctionnalités en une seule fois.

Tu dois travailler en **\*\*vertical slices\*\***.

À chaque fonctionnalité, tu dois laisser le dépôt dans un état stable et compilable.

Tu dois privilégier :

- robustesse ;
- sécurité ;
- simplicité ;
- réutilisabilité ;
- performance ;
- UX ;
- accessibilité ;
- offline-first ;
- maintenabilité.

Si une décision architecturale importante est nécessaire, arrête l'implémentation et documente la décision avant de poursuivre.

Si une fonctionnalité existante est incomplète ou dangereuse, corrige-la avant de construire dessus.

Le but n'est pas de produire beaucoup de code.

Le but est de produire un **\*\*produit réellement utilisable, sécurisé, commercialisable et évolutif\*\***.

---

## PREMIÈRE MISSION DE L'AGENT

Commencer par un audit technique du dépôt.

Ne modifier aucun code immédiatement.

Produire :

1. architecture actuelle ;

2. modules backend ;
3. modules frontend ;
4. architecture Flutter ;
5. modèle de données ;
6. endpoints ;
7. authentification/autorisation ;
8. multi-tenant ;
9. offline ;
10. notifications ;
11. intégrations ;
12. tests ;
13. dette technique ;
14. risques ;
15. fonctionnalités déjà présentes mais incomplètes ;
16. fonctionnalités réellement opérationnelles ;
17. plan de migration vers la roadmap des 20 fonctionnalités.

Ensuite seulement, proposer la première vertical slice à implémenter.

**\*\*FIN DU PROMPT\*\***